Documentation API

22/02/2025 - Version 0.26.0

Important Note for dear readers: As this is currently a Beta version, certain APIs documented here are subject to change in future versions of their respective products.

Important Note for IO Developers: If you update this document, please make sure to:

- make a copy of previous version for archives
- update the change history below
- increase version + date of this document

API Principles **5**
HTTP Verbs 5
HTTP Status Codes 5
Error Responses 6
Language 6
API Authentication **7**
User Model 7
Request an Access Token 9
Save the access token 12
Access the API 12
Access Token Expiration 12
Product information management (PIM) **13**
Overview 13
Menu Model 14
Category Model 14
CategoryProduct Model 14
Product Model 14
ProductStep Model 15
Step Model 15
StepProduct Model 16
Tag Model 16
Get the compressed menu (hashmap-based) 17
CRUD Menu 18
Get menu by restaurantId 18
Create a menu by restaurantId 18
Import a JSON Menu 19
CRUD Category 20
Get a category 20
Create a category 20
Delete a category 20
Update a category 21
CRUD Product 21
Get a product 21
Create a product 21
Delete a product 21
Update a product 22
CRUD Step 22
Get a step 22
Create a step 22
Delete a step 22
Update a step 22

CRUD Tags 23
Get a tag 23
Create a tag 23
Delete a tag 23
Update a tag 23
Images library 25
Overview 25
Image Model 25
Get the images 25
Create one image 26
Get one image 26
Update one image 26
Delete one image 27
Customers 28
Customer Model 28
Find all customers 29
Get all customer orders 30
Forgotten password 31
Create a forgotten password token 31
Change password of a customer using forgotten password token 31
Create a customer 31
E-wallet 33
Get all badges 33
Get the e-wallet balance of a customer 33
Transaction Model 34
Get all e-wallet transactions for a customer 37
Credit/debit an e-wallet 39
Affluence 41
Get the number of orders between two dates 41
Get the number of orders by interval between two dates 42
Order Management System (Web+Kiosk) 44
Order constants 44
Consumption modes 44
Order sources 44
Order main status 44
Order payment status 44
Order delivery status 44
Order model 45
Preview an order 53
Order preview errors 58
Basic Errors 59
Core Errors 61
Cart Errors 62
PaymentMethod Errors 66

Schedule Errors 67
Discount Errors 68
Create an order 70
Couponing **72**
Create a coupon 72
Use a coupon on webordering 73
Overview Analytics **73**

API Principles

HTTP Verbs

Different HTTP verbs (GET, POST, etc.) are used for different interactions with the API. Most of the time the required verbs for a particular operation will be included in documentation, but it's helpful to have a general understanding of how the verbs are used.

- POST - create new resources (Orders, products, users, etc.)
- GET - read resources (catalog, data, etc.)
- PUT - update existing resources (orders, products, etc.)
- DELETE - delete resources (users, products, etc.)

HTTP Status Codes

Overview HTTP status codes are an important piece of metadata offered by the API and can be used to determine the outcome of an API request. By inspecting the status code, which is included as an HTTP header and in the API response, you can determine if there was a problem with the request, and what the type of problem was (authorization, resource not found, etc.).

All API responses are classified using one of the following response types:

- **Successful 2xx -** This class of response codes indicates success.
  - **200 - OK** - success, and a resource is included in the response; typically returned by GET requests
  - **201 - Created** - success, and the created resource is included in the response; typically returned by POST requests
  - **204 - No Content** - success, however no other content is contained in the response; typically returned by PUT and DELETE requests
- **Client Error 4xx -** This class of response codes indicates that your client has made a mistake, for example, a 404 code indicates that the specified resource wasn't found. These

types of errors are typically recoverable (e.g. try the request again with the correct URL).
    - **400 - Bad Request** - general error status code indicating that your client did something wrong
    - **401 - Unauthorized** - the request requires user authentication; try the request again, making sure your OAuth parameters are set correctly
    - **403 - Forbidden** - the client does not have the privileges required to access the resource
    - **404 - Not Found** - the requested resource was not found
- **Server Error 5xx -** This class of response codes indicates that an internal server issue occurred (e.g. the server was not able to connect to a backend database).
    - **500 - Internal Server Error** - an internal server error occurred 2. 503 - Service Unavailable - the server is overloaded and can't process the request

Error Responses

When an error occurs, the API will attempt to provide you with as much detail as possible regarding what went wrong. Here's what a typical API error response looks like:

```
{"status":401,"code":"permission_denied","message":"Vous n'avez pas la permission pour accéder à cette ressource."}
```

The error response contains a status code that matches the HTTP status code from the HTTP header - it's just included in the response for convenience. The API error response also returns an error code which can be used later in order to track down an issue

Language

The default language of the API is **english**. The supported languages are **english** and **french**.

You can specify the language by setting the parameter **io-locale** in the headers of a request. This language will be used to translate the message returned by the API (response message, errors, payload, files) but could also be used by other services later.

For example:

> If an order is created with the hearder **io-locale** set to “fr”. If the customer should receive a mail, it will be in french. And all next mails related to this order will be in french.

API Authentication

Note: The following examples are presented in curl. Feel free to adapt these examples to your favorite programming language.

User Model

```
userId: {
    type: DataTypes.BIGINT,
    field: 'user_id',
    allowNull: false,
    primaryKey: true,
    autoIncrement: true,
},
firstName: {
    type: DataTypes.STRING,
    field: 'first_name',
    allowNull: true,
},
lastName: {
    type: DataTypes.STRING,
    field: 'last_name',
    allowNull: true,
},
phone: {
    type: DataTypes.STRING,
    allowNull: true,
    validate: {
        is: {
            args: [new RegExp(regex.phone)],
            msg: 'phone_not_valid',
        },
    },
},
email: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
        isEmail: {
            args: [true],
            msg: 'email_not_valid',
        },
    },
},
```

```
password: {
    type: DataTypes.STRING,
    allowNull: false,
},
passwordSalt: {
    type: DataTypes.STRING,
    field: 'password_salt',
    allowNull: false,
    defaultValue: false,
},
passwordService: {
    type: DataTypes.STRING,
    field: 'password_service',
    allowNull: true,
},
role: {
    type: DataTypes.STRING,
    allowNull: false,
},
forgottenPasswordToken: {
    field: 'forgotten_password_token',
    type: DataTypes.STRING,
    allowNull: true,
},
forgottenPasswordTokenDate: {
    field: 'forgotten_password_token_date',
    type: DataTypes.DATE,
    allowNull: true,
},
additionalAccessFlags: {
    type: DataTypes.TEXT,
    field: 'additional_access_flags',
    allowNull: true,
},
disallowedAccessFlags: {
    type: DataTypes.TEXT,
    field: 'disallowed_access_flags',
    allowNull: true,
},
lastConnection: {
    field: 'last_connection',
    type: DataTypes.DATE,
    allowNull: true,
},
brandId: {
    type: DataTypes.BIGINT,
    field: 'brand_brand_id',
```

```
    allowNull: true,
},
accessFlags: {
    type: DataTypes.VIRTUAL, // concatenate the default accessFlags of the role + additionalAccessFlags - disallowedAccessFlags
},
```

Request an Access Token

First, you need to have a working account email/password with the right access to the resources you want to query. You can then request an access_token. An access_token is essentially a long-lived token that will allow your client to call API methods.

Access tokens issued by the API are long lived, so just request one and save it to use for future API calls. Once you get an access_token, you will include it in all future API requests. Use your email and password:

POST https://api.innovorder.fr/oauth/login

Body(x-www-form-urlencoded):

```
rememberMe=true
username=myEmail@emailProvider.com
password=myPassword
grant_type=password
```

- **rememberMe(boolean)**: if true the expiration date of the token is set to a week, else it is a day
- **username(str)**: email of the account
- **password(str)**: password of the account
- **grant_type(str)**: must be set to password

Response:


```
{
    "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhY2Nlc3NUb2tlbiI6IjVjNTQyOGQ2ZjgyNGQxNmIzYjdlYmVjZDlmODNiYzQxYzk1MzdkZjY0Nzc2OTIzMjdmMzA5MzJjOTJhYzZlMTciLCJyZWZyZXNoVG9rZW4iOiI1MTFhZTk2ZTliMjM3MGJkNjhjZmQ5NWUzN2RjNGYxMDIxNjdiZDVjNWVlMm
```

```
Q1NGQ2NmMwZjFhNDNlMjBhYzQwIiwiYWNjZXNzVG9rZW5FeHBpcmF0aW9uRGF0ZSI6IjIwMTktMT
ItMTlUMTM6MjU6MjkuNDA4WiIsInVzZXJJZCI6MTAyOCwiY3VzdG9tZXJJZCI6bnVsbCwicmVtZW
1iZXJNZSI6dHJ1ZSwiaWF0IjoxNTc1NTUyMzI5fQ.v1hmnnUfaCTsm3KyzDffobDgKQKnuj79_uV
7rqIm9PA",
    "status": 201,
    "code": "token_created",
    "message": "You have successfully logged in.",
    "data": {
        "user": {
            "accessFlags": {
                "CAN_ARCHIVE_CUSTOMERS": true,
                "CAN_FORCE_DEFAULT_CREDENTIALS_CHANGE": true,
                "CAN_EDIT_CUSTOMERS_CREDENTIALS": true,
                "CAN_EXPORT_CUSTOMERS_CSV": true,
                "CAN_EDIT_AREAS": true,
                "CAN_CREATE_MENU_ITEMS": true
            },
            "userId": 1,
            "firstName": "FirstName",
            "lastName": "LastName",
            "phone": "0123456789",
            "email": "myEmail@emailProvider.com",
            "password":
"d8a75113148d60efee799c03d9a35f00917d02ccdc734f3b6370504df19748aa",
            "passwordSalt":
"8873e6f51eab24c994837932385014b0f3d1e1ced9f2cf4f45cd9b8bd36337f3",
            "passwordService": null,
            "role": "brand",
            "forgottenPasswordToken": null,
            "forgottenPasswordTokenDate": null,
            "additionalAccessFlags":
"CAN_ARCHIVE_CUSTOMERS,CAN_FORCE_DEFAULT_CREDENTIALS_CHANGE,CAN_EDIT_CUSTOME
RS_CREDENTIALS",
            "disallowedAccessFlags": "",
            "lastConnection": "2019-12-05T13:25:29.233Z",
            "brandId": 10,
            "created_at": "2017-01-31T17:09:04.000Z",
            "updated_at": "2019-12-05T13:25:29.000Z",
            "deleted_at": null,
            "brands": [
                {
                    "brandId": 10,
                    "name": "Brand Name"
                }
            ],
            "restaurants": [
                {
```

```
                    "restaurantId": 11,
                    "name": "Restaurant Name",
                    "address": {
                        "addressId": 480,
                        "lat": 48.868171,
                        "lng": 2.373191,
                        "timeZoneId": "Europe/Paris",
                        "locality": "Paris",
                        "route": "Rue Blondel",
                        "postalCode": "75011",
                        "streetNumber": "30",
                        "administrativeAreaLevel1": "Île-de-France",
                        "administrativeAreaLevel2": "2nd arrondissement of
Paris",
                        "country": "fr",
                        "googlePlaceId": "ChIJ3e6Ykgdu5kcRJXtvh-3JyLU",
                        "providerId": "address.902052405",
                        "phone": null,
                        "additionalInfo": null,
                        "digiCode": null,
                        "apartmentNumber": null,
                        "iconIdentifier": null,
                        "customerId": null,
                        "name": "30 Rue Blondel 75002 Paris, France",
                        "created_at": "2017-01-31T16:28:02.000Z",
                        "updated_at": "2019-11-15T12:39:08.000Z",
                        "deleted_at": null
                    }
                }
            ]
        },
        "tokenType": "Bearer",
        "accessToken":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhY2Nlc3NUb2tlbiI6IjVjNTQyOGQ2ZjgyNG
QxNmIzYjdlYmVjZDlmODNiYzQxYzk1MzdkZjY0Nzc2OTIzMjdmMzA5MzJjOTJhYzZlMTciLCJyZW
ZyZXNoVG9rZW4iOiI1MTFhZTk2ZTliMjM3MGJkNjhjZmQ5NWUzN2RjNGYxMDIxNjdiZDVjNWVlMm
Q1NGQ2NmMwZjFhNDNlMjBhYzQwIiwiYWNjZXNzVG9rZW5FeHBpcmF0aW9uRGF0ZSI6IjIwMTktMT
ItMTlUMTM6MjU6MjkuNDA4WiIsInVzZXJJZCI6MTAyOCwiY3VzdG9tZXJJZCI6bnVsbCwicmVtZW
1iZXJNZSI6dHJ1ZSwiaWF0IjoxNTc1NTUyMzI5fQ.v1hmnnUfaCTsm3KyzDffobDgKQKnuj79_uV
7rqIm9PA"
    },
    "token_type": "Bearer"
}
```

Save the access token

Make special note of the access_token field. You'll notice that some of the values are repeated under different key names - they can be safely ignored if they are not mentioned in the specification.

Access the API

You can use the access_token to make API requests:

GET https://api.innovorder.fr/oauth/me

Headers:

- Authorization: Bearer <token>

Response:

same as /oauth/login

Access Token Expiration

There currently isn't an explicit expiration policy for access tokens. Therefore, it's recommended that you wrap your API code in an error handling block (like a try/catch) and watch for a 401 Unauthorized HTTP status code. You can then request a new access token and resend your original API request.

Product information management (PIM)

Overview

The menu is based on the following entities:

- Category and sub-category: for grouping products
- Product: represents an item that can be bought. It contains steps.
- Step: represents the option of an item. It contains products.
- Tag: you can tag a product. It is commonly used for analytics purposes or for splitting an order between multiple printers.

The menu is the complete catalog including categories, sub-categories, products, steps and tags.

A product is a bi-recursive data structure between products and steps. A product is the entry of the structure, then we have steps, then we have products, etc.

Example:

Menu Model

```
export interface MenuAttributes {
   menuId: number;
   restaurantId: number | null; // only one of restaurantId or brandId is
defined
   brandId: number | null;
   is_enabled: boolean;
   name: string | null;
   hasInitModalBeenShowed: boolean;
   created_at: Date;
   updated_at: Date;
}
```

Category Model

```
 export interface CategoryAttributes {
   categoryId: number;
   name: string;
   imageId: number | null;
   position: number;
   parentCategoryId: number | null;
   menuId: number;
   webAvailability: boolean;
   kioskAvailability: boolean;
}
```

CategoryProduct Model

```
export interface CategoryProductAttributes {
   categoryId: number;
   menuId: number;
   productId: number;
   position: number;
   created_at: Date;
   updated_at: Date;
}
```

Product Model

```
export interface ProductAttributes {
   productId: number;
   menuId: number;
   signature: number;
   version: number;
   name: string;
   description: string | null;
```

```
    additionalInformation: string | null;
    price: number;
    isFormula: boolean | null;
    webAvailability: boolean;
    kioskAvailability: boolean;
    vat: number;
    isUpselling: boolean | null;
    isFinalProduct: boolean | null;
    consumptionFlags: Flags;
    isProducible: boolean | null;
    currentQuantity: number | null;
    defaultQuantity: number | null;
    isSitInEnabled: boolean | null; // Deprecated: consumptionFlags is used
instead
    isDeliveryEnabled: boolean | null; // Deprecated: consumptionFlags is used
instead
    isTakeAwayEnabled: boolean | null; // Deprecated: consumptionFlags is used
instead
    imageId: number | null;
    canApplyGrant: boolean | null;
    canApplyEntranceFee: boolean | null;
    servingCount: number;
    isEligibleLuncheon: boolean | null;
    created_at: Date;
    updated_at: Date;
    deleted_at: Date | null;
}
```

ProductStep Model

(steps included in a product)

```
export interface ProductStepAttributes {
    productStepId: number;
    stepId: number;
    menuId: number;
    productId: number;
    position: number;
    created_at: Date;
    updated_at: Date;
}
```

Step Model

```
export interface StepAttributes {
    stepId: number;
    signature: string;
    version: number;
```

```
    name: string;
    ticketLabel: string;
    minChoice: number;
    maxChoice: number;
    isQuantitySelectionEnabled: boolean;
    isWeightStrategyEnabled: boolean;
    isImageDisplayEnabled: boolean;
    isDescriptionDisplayEnabled: boolean;
    isGroupEnabled: boolean;
    isExcessAllowed: boolean;
    menuId: number;
    webAvailability: boolean;
    kioskAvailability: boolean;
    created_at: Date;
    updated_at: Date;
    deleted_at: Date | null;
}
```

StepProduct Model

(products included in a step)

```
export interface StepProductAttributes {
    stepProductId: number;
    stepId: number;
    menuId: number;
    productId: number;
    position: number;
    price: number;
    weight: number;
    exceedPrice: number;
    groupName: string;
    maxProductQuantity: number;
    created_at: Date;
    updated_at: Date;
}
```

Tag Model

```
export interface TagAttributes {
    tagId: number;
    brandId: number | null;
    name: string;
    color: string | null;
    is_enabled: boolean;
    created_at: Date | null;
    updated_at: Date | null;
}
```

Get the compressed menu (hashmap-based)

GET https://api.innovorder.fr/menus/<menuId>/hash_map

Description

This will provide the full menu for a given <menuId>


```
▼{status: 200, code: "menu_hash_map_succeed", message: "Vous avez accès à ces menus.",…}
  code: "menu_hash_map_succeed"
  ▼data: {layout: [{categoryId: 38885, webAvailability: true, kioskAvailability: true, image: null},…],…}
    ▼categories: {,…}
      ▶38885: {categoryId: 38885, name: "Menus", imageId: null, position: 0, parentCategoryId: null, menuId: 2316,…}
      ▶38886: {categoryId: 38886, name: "Plats", imageId: null, position: 1, parentCategoryId: null, menuId: 2316,…}
      ▶38887: {categoryId: 38887, name: "Suppléments", imageId: null, position: 2, parentCategoryId: null,…}
      ▶38888: {categoryId: 38888, name: "Desserts", imageId: null, position: 3, parentCategoryId: null, menuId: 2316,…}
      ▶38889: {categoryId: 38889, name: "Boissons", imageId: null, position: 4, parentCategoryId: null, menuId: 2316,…}
    ▼layout: [{categoryId: 38885, webAvailability: true, kioskAvailability: true, image: null},…]
      ▶0: {categoryId: 38885, webAvailability: true, kioskAvailability: true, image: null}
      ▶1: {categoryId: 38886, webAvailability: true, kioskAvailability: true, image: null}
      ▶2: {categoryId: 38887, webAvailability: true, kioskAvailability: true, image: null}
      ▶3: {categoryId: 38888, webAvailability: true, kioskAvailability: true, image: null}
      ▶4: {categoryId: 38889, webAvailability: true, kioskAvailability: true, image: null}
    ▶products: {464462: {productId: 464462, signature: 429353, version: 1, name: "A point", description: null,…},…}
    ▶steps: {,…}
    message: "Vous avez accès à ces menus."
    status: 200
```

CRUD Menu

Get menu by restaurantId

GET
https://api.innovorder.fr/menus?restaurantId=<restaurantId>

Description
Find all menus for a given restaurantId

Create a menu by restaurantId

POST
https://api.innovorder.fr/menus

Description
Create a menu

```
{
   name: string;
   channels: number[];
   restaurantId: number;
}
```

DELETE
https://api.innovorder.fr/menus/:menuId
channelWEB:2
Description
Delete a menu

PUT
https://api.innovorder.fr/menus/:menuId

Description
Update a menu

```
{
   name: string;
   channels: number[];
```

```
  restaurantId: number;
  menuId: number;
}
```

Import a JSON Menu

POST

https://api.innovorder.fr/menus

```

  name: string;
  channels: number[];
  restaurantId: number;
  import: FlattenedInstancesMenu;
}
```

where `FlattenedInstancesMenu` is:

```
FlattenedInstancesMenu = {

  layout: number[];

  categories: Record<number, CategoryInstance>;

  categoryProducts: Record<number, CategoryProductInstance>;

  products: Record<number, ProductInstance>;

  productSteps: Record<number, ProductStepInstance>;

  steps: Record<number, StepInstance>;

  stepProducts: Record<number, StepProductInstance>;

  tags: Record<number, TagInstance>;

  images: Record<number, ImageInstance>;

};
```

Please see previous models for description of fields.

CRUD Category

Get a category

GET
https://api.innovorder.fr/menus/<menuId>/categories/<categoryId>

Description
Provide information of a given category by its <categoryId> for a given <menuId>

Create a category

POST
https://api.innovorder.fr/menus/<menuId>/categories

Description
Create a category by its <categoryId> for a given <menuId>
Body

```
{
 menuId: number;
 name: string;
 position: number;
 parentCategoryId: number | null;
 imageId: number | null;
}
```

Delete a category

DELETE
https://api.innovorder.fr/menus/<menuId>/categories/<categoryId>

Description
Delete a given category by its <categoryId> for a given <menuId>

Update a category

PUT
https://api.innovorder.fr/menus/<menuId>/categories/<categoryId>

Description
Update a given category by its <categoryId> for a given <menuId>

CRUD Product

Get a product

GET
https://api.innovorder.fr/menus/<menuId>/products/<productId>

Description
Provide information of a given product by its <productId> for a given <menuId>

Create a product

POST
https://api.innovorder.fr/menus/<menuId>/products

Description
Create a product by its <productId> for a given <menuId>

Delete a product

DELETE
https://api.innovorder.fr/menus/<menuId>/products/<productId>

Description
Delete a given product by its <productId> for a given <menuId>

Update a product

PUT
https://api.innovorder.fr/menus/<menuId>/products/<productId>

Description
Update a given product by its <productId> for a given <menuId>

CRUD Step

Get a step

GET
https://api.innovorder.fr/menus/<menuId>/steps/<stepId>

Description
Provide information of a given by its <stepId> for a given <menuId>

Create a step

POST
https://api.innovorder.fr/menus/<menuId>/steps

Description
Create a step for a given <menuId>

Delete a step

DELETE
https://api.innovorder.fr/menus/<menuId>/steps/<stepId>

Description
Delete a given step by its <stepId> for a given <menuId>

Update a step

PUT

https://api.innovorder.fr/menus/<menuId>/steps/<stepId>

Description
Update a given step by its <stepId> for a given <menuId>

CRUD Tags

Get a tag

GET
https://api.innovorder.fr/menus/<menuId>/tags/<tagId>

Description
Provide information of a given tag by its <tagId> for a given <menuId>

Create a tag

POST
https://api.innovorder.fr/menus/<menuId>/tags

Description
Create a tag for a given <menuId>

Delete a tag

DELETE
https://api.innovorder.fr/menus/<menuId>/tags/<tagId>

Description
Delete a given tag by its <tagId> for a given <menuId>

Update a tag

PUT
https://api.innovorder.fr/menus/<menuId>/tags/<tagId>

Description

Update a given step by its <tagId> for a given <menuId>

Images library

Overview

The images of the menu are registered in a library. An image could be scoped to a brand or to a restaurant.

A brand image is available for the menus of the brand and all the menus of all the restaurants of the brand

A restaurant image is only available for the menus of its restaurant

Image Model

```
export interface ImageAttributes {
   imageId: number;
   filename: string | null;
   originalFilename: string | null;
   fullUrl: string;
   absoluteFilePath: string;
   size: number;
   mimeType: string;
   created_at: Date;
   updated_at: Date;
   focalX: number;
   focalY: number;
   zoom: number;
   restaurantId: number | null;
   brandId: number;
}
```

Get the images

Brand images

GET https://api.innovorder.fr/library/brand/<brandId>

QueryParams:

- **limit (optional)(number)**: specify the maximum number of orders which will be returned
- **offset (optional)(number)**: specify the number of orders which will be skipped.

Pagination: Use limit + offset to paginate your results.

Restaurant image

GET https://api.innovorder.fr/library/restaurant/<restaurantId>

QueryParams:

- **limit (optional)(number)**: specify the maximum number of orders which will be returned
- **offset (optional)(number)**: specify the number of orders which will be skipped.

Pagination: Use limit + offset to paginate your results.

Create one image

Brand image

POST https://api.innovorder.fr/library/brand/<brandId>?resize=true

+ the binary of file image in the formdata

This return an image instance

Restaurant image

POST https://api.innovorder.fr/library/restaurant/<restaurantId>?resize=true

+ the binary of file image in the formdata

This return an image instance

Get one image

There is no route to get one image, the images are always returned with their associated instances

Update one image

Brand images

1. Upload the image file

PUT https://api.innovorder.fr/library/amend/brand/<brandId>?resize=true

+ the binary of file image in the formdata

This doesn't link the image file to any image instance

2. Update the image instance

PUT https://api.innovorder.fr/library/brand/<imageId>

Body:

- Required:

imageId: number;

fileName: string;

- Optional:

  The other image attributes

Use the result of the upload request

Restaurant images

1. Upload the image file

PUT https://api.innovorder.fr/library/amend/restaurant/<restaurantId>?resize=true

+ the binary of file image in the formdata

This doesn't link the image file to any image instance

2. Update the image instance

PUT https://api.innovorder.fr/library/restaurant/<imageId>

Body:

- Required:

  imageId: number;

  fileName: string;

- Optional:

  The other image attributes

Use the result of the upload request

Delete one image

Brand image

DELETE https://api.innovorder.fr/library/brand/<imageId>

Restaurant image

DELETE https://api.innovorder.fr/library/restaurant/<imageId>

Customers

Customers are end-users performing actions on our platform (ordering, using e-wallet, etc.)

Customer Model

```
paymentDailyLimit: {
    type: DataTypes.BIGINT,
    allowNull: true,
},
ewalletOverdraftAmount: {
    type: DataTypes.BIGINT,
    allowNull: true,
},
brandId: {
    type: DataTypes.BIGINT,
    allowNull: true,
},
archivedAt: {
    type: DataTypes.DATE,
    defaultValue: null,
    allowNull: true,
},
disabledAt: {
    type: DataTypes.DATE,
    defaultValue: null,
    allowNull: true,
},
ewalletReloadRecieveConfirmation: {
    type: DataTypes.BOOLEAN,
    allowNull: false,
    defaultValue: false,
},
ewalletReloadConfirmationEmail: {
    type: DataTypes.STRING,
    allowNull: true,
}
```

Find all customers

GET https://api.innovorder.fr/customers?brandId=<brandId>

QueryParams:

- **brandId (mandatory)(number)**: <brandId> is found when doing a **GET** on **/oauth/me** or during the login process.
- **include (optional)(string)**: customers related fields/data which could be included. List separated by "," from:
  - orderStats: add totalOrders, totalSpent, averagePurchase, lastOrder and averageOrderPerWeek attributes to each customer
  - ewalletBalance: add the ewalletBalance to eachCustomer
- **restaurantIds(optional)(string)**: filter the customers from specifics restaurants. Ids listed and separated by ",". (default is all restaurants of the brand)
- **unitCurrency(optional)(boolean)**: return the ewalletBalance and the loyaltyBalance in a float string corresponding to the currency used. (ex 2€50 are returned 250 by default and will be returned "2.50" with unitCurrency)
- **mustDisplayArchivedCustomers(optional)(boolean)**: exclude the customers without badgeNumber (default is false)
- **mustHaveBadgeNumber(optional)(boolean)**: include the archived and disabled customers (default is false)
- **firstName(optional)(string)**: filter the customers by exact firstName (case insensitive) (Café = cafe but caf != cafe)
- **lastName(optional)(string)**: filter the customers by exact lastName (case insensitive)
- **email(optional)(string)**: filter the customers by exact email (case insensitive)
- **badgeNumber(optional)(string)**: filter the customers by exact badgeNumber (case sensitive) (Café != cafe && caf != cafe)
- **balance(optional)(number)**: filter the customers by exact e-wallet amount
- **search(optional)(string)**: filter the customers by partial matching on firstName or lastName or phone or studentNumber, badgeNumber or email
- **studentNumber(optional)(string)**: filter the customers by exact guest number(case insensitive)

Headers:

- Authorization: Bearer <token>

Response:

```
{
   "status": 200,
   "code": "customers_succeed",
   "message": "Vous pouvez accéder à ces clients",
   "data": [
       {
           "customerId": 1,
           "firstName": "FirstName",
           "lastName": "LastName",
           "email": "test@innovorder.fr",
           "tripAdvisorUsername": null,
           "birthdate": null,
           "badgeNumber": null,
           "disabledAt": null,
           "archivedAt": null,
           "studentNumber": null,
           "class": null,
           "section": null,
           "subSection": null,
           "tariffCode": null,
           "grantCode": null,
           "entranceFeeCode": null,
           "barCode": null,
           "pricingRuleCode": null,
           "phone": "0123456789
       }
   ]
}
```

Get all customer orders

See Tickets section (Get the orders subsection): use customerId filter.
E.g: GET /orders?brandId=<brandId>&customerId=<customerId>

Forgotten password

This is a two-step process:

1. Generate a forgotten password token for a given customer using its email address
2. Use the forgotten password token with the customerId to change the password of the customer

Create a forgotten password token

POST https://api.innovorder.fr/customers/forgotten_password

Headers:

- Authorization: Bearer <token>

Body:

- Email: email of the customer

Change password of a customer using forgotten password token

POST https://api.innovorder.fr/customers/<customerId>/forgotten_password/<forgottenPasswordToken>

Headers:

- Authorization: Bearer <token>

Body:

- password: new password for this user

Create a customer

POST https://api.innovorder.fr/<brandId>/customers

Headers:

- Authorization: Bearer <token>

Body:

```
{
"role": "customer",
"brandId": <brandId>,
"badgeNumber": <generatedBadgeNumber>,
"firstName": <firstNameOfTheCustomer>,
```

```
"lastName": <lastNameOfTheCustomer>,
"phone": <phoneNumberOfTheCustomer - can be set to 0102030405 if not available>,
"email": <emailOfTheCustomer>, // can be set to a random email address
"password": <passwordOfTheCustomer>,
"confirmPassword": <passwordOfTheCustomer>,
"student": { "accountPaymentType": "PREPAYMENT", "accountType": null }
}
```

E-wallet

The “e-wallet” is a wallet technology we’ve created at Innovorder. User can credit/debit digitally (using a web interface) or physically (on an integrated hardware).

Get all badges

You can use the customer ressource for getting all the badges by calling:

GET https://api.innovorder.fr/customers?brandId=<brandId>&include=orderStats,optIn,ewalletBalance

The badge number is under the “badgeNumber” field.

You can also use the following endpoint which is paginated:

GET /customers/information_ewallet?limit=500&offset=30

Get the e-wallet balance of a customer

GET https://api.innovorder.fr/customers/<customerId>/balance?unscoped=true

Headers:

- Authorization: Bearer <token>

Response:

```
{
  "status":200,
  "code":"balance_succeed",
  "message":"Vous avez accès à ce contenu.",
  "data":{
    "customerBalance":2764
  }
}
```

- **customerBalance**: integer, final balance in cents for a given customer

Transaction Model

```
transactionId: {
    type: DataTypes.BIGINT,
    allowNull: false,
    field: 'transaction_id',
    primaryKey: true,
    autoIncrement: true,
},
uniqueTransactionId: {
    type: DataTypes.STRING,
    allowNull: false,
    field: 'unique_transaction_id',
    defaultValue: () => uuidV4(),
},
amount: {
    type: DataTypes.INTEGER,
    allowNull: false,
},
error: {
    type: DataTypes.TEXT('long'),
    allowNull: true,
},
type: {
    type: DataTypes.ENUM('1', '2', '3', '4', '5'),
    allowNull: false,
},
status: {
    type: DataTypes.ENUM('1', '2', '3', '4', '5', '6'),
    allowNull: false,
},
remoteAddress: {
    type: DataTypes.STRING,
    field: 'remote_address',
    allowNull: true,
},
source: {
    type: DataTypes.ENUM('1', '2', '3', '4', '5', '6', '7'),
    allowNull: false,
},
currency: {
    type: DataTypes.STRING,
    field: 'currency',
    allowNull: false,
},
externalUserReference: {
```

```
    type: DataTypes.STRING,
    field: 'external_user_reference',
    allowNull: true,
},
externalUserReferenceId: {
    type: DataTypes.STRING,
    field: 'external_user_reference_id',
    allowNull: true,
},
externalOrderReference: {
    type: DataTypes.STRING,
    field: 'external_order_reference',
    allowNull: true,
},
externalDailyOrderId: {
    type: DataTypes.STRING,
    field: 'external_daily_order_id',
    allowNull: true,
},
externalDeviceId: {
    type: DataTypes.STRING,
    field: 'external_device_id',
    allowNull: true,
},
sourceUserId: {
    type: DataTypes.BIGINT,
    field: 'source_user_source_user_id',
    allowNull: true,
},
originTransactionId: {
    type: DataTypes.BIGINT,
    field: 'origin_transaction_id',
    allowNull: true,
    unique: 'origin_transaction_index',
},
paymentOrigin: {
    type: DataTypes.ENUM('1', '2', '3', '4', '5', '6', '7'),
    field: 'payment_origin',
    allowNull: true,
},
paymentOriginMetaData: {
    type: DataTypes.STRING,
    field: 'payment_origin_meta_data',
    allowNull: true,
},
brandId: {
    type: DataTypes.BIGINT,
```

```
    field: 'brand_brand_id',
    allowNull: false,
},
restaurantId: {
    type: DataTypes.BIGINT,
    field: 'restaurant_restaurant_id',
    allowNull: true,
},
customerId: {
    type: DataTypes.BIGINT,
    field: 'customer_customer_id',
    allowNull: true,
},
userId: {
    type: DataTypes.BIGINT,
    field: 'user_user_id',
    allowNull: true,
},
orderId: {
    type: DataTypes.BIGINT,
    field: 'order_order_id',
    allowNull: true,
},
groupTransactionId: {
    type: DataTypes.STRING,
    field: 'group_transaction_id',
    allowNull: true,
},
loyaltyCardId: {
    type: DataTypes.BIGINT,
    field: 'loyalty_card_loyalty_card_id',
    allowNull: true,
},
applyDate: {
    type: DataTypes.DATE,
    field: 'apply_date',
    allowNull: false,
}
```

Corresponding enums:

```
status: {
 "1": "OK",
 "2": "REFUSED",
 "3": "CANCELLED",
 "4": "ERROR",
 "5": "PENDING",
 "6": "REQUIRES_ACTION"
}
```

```
source: {
 "1": "DASHBOARD",
 "2": "CEL",
 "3": "EWALLET_RELOADER",
 "4": "RECEPTION",
 "5": "POS",
 "6": "KIOSK",
 "7": "TIME_CHEF_API"
}
type: {
 "1": "CREDIT",
 "2": "REFUND",
 "3": "DEBIT",
 "4": "CANCEL",
 "5": "DATE_MODIFICATION"
}
paymentOrigin: {
 "1": "CASH",
 "2": "CHECK",
 "4": "CREDIT_CARD_ONLINE",
 "5": "CREDIT_CARD",
 "6": "LUNCHEON_VOUCHER",
 "7": "TIME_CHEF",
 "3": "OTHER"
}
```

Get all e-wallet transactions for a customer

GET https://api.innovorder.fr/customers/<customerId>/balance_history

QueryParams:

- **unscoped (optional)(boolean)**: if true it enables to query on archived or deleted customer. Default is false.
- **include (optional)(string)**: The only value available is `include=order`. It adds the orders to the transactions if they exist.
- **order (optional)(string)**: order the transaction by an attributes. Exemple: `order=-createdAt` to obtain the last transactions first.
- **limit (optional)(number)**: specify the maximum number of orders which will be returned
- **offset (optional)(number)**: specify the number of orders which will be skipped.

Pagination: Use limit + offset to paginate your results.

Headers:

- Authorization: Bearer <token>

Response:


```
{
  "status":200,
  "code":"balance_succeed",
  "message":"Vous avez accès à ce contenu.",
  "data":[
     {
        "transactionId":2689871,
        "type":"1",
        "currency":"EUR",
        "amount":14,
        "paymentOrigin":"3",
        "paymentOriginMetaData":"Metadata...",
        "customerId":277957,
        "applyDate":"2019-10-01T15:47:42.000Z",
        "createdAt":"2019-10-01T15:47:42.000Z",
        "externalOrderReference":null,
        "orderTicketNumber":null,
        "orderId":null,
        "restaurantName":null,
        "reportOrderRestaurantName":null,
        "deviceName":null,
        "externalUserReference":null,
        "error":null,
        "source":"5",
        "sourceUserEmail":null,
        "sourceUserFirstName":null,
        "sourceUserLastName":null,
        "sourceUserId":null,
        "status":"1"
     },
     {
        "transactionId":2997781,
        "type":"3",
        "currency":"EUR",
        "amount":-572,
        "paymentOrigin":null,
        "paymentOriginMetaData":null,
        "customerId":277957,
        "applyDate":"2019-11-14T11:27:26.000Z",
        "createdAt":"2019-11-14T11:27:26.000Z",
        "externalOrderReference":"xxx-yyy-zzz",
        "orderTicketNumber":null,
        "orderId":null,
        "restaurantName":null,
        "reportOrderRestaurantName":"xxx",
        "deviceName":"POS Master",
        "externalUserReference":"Gérant",
```

```
        "error":null,
        "source":"5",
        "sourceUserEmail":"user.test@innovorder.fr",
        "sourceUserFirstName":null,
        "sourceUserLastName":null,
        "sourceUserId":69003,
        "status":"1"
      }
    ]
  }
```


Credit/debit an e-wallet

POST https://api.innovorder.fr/customers/transactions

Headers:

- Authorization: Bearer <token>

Body (x-www-form-urlencoded):


```
"customerId":130069,
"type":"1",
"brandId":262,
"source":"7",
"amount":"200",
"paymentOrigin":"1",
"paymentOriginMetaData":" Test"
```


- **brandId (int)**: destination brand of the transaction
- **customerId (int)**: destination customer of the transaction
- **type (str)**: “1” for “CREDIT” or “3” for “DEBIT”
- **source (str)**: “1” for “DASHBOARD”, see ENUM below:


```
{
  "DASHBOARD": "1",
  "CEL": "2",
```

```
  "EWALLET_RELOADER": "3",
  "RECEPTION": "4",
  "POS": "5",
  "KIOSK": "6",
  "TIME_CHEF_API": "7",
  "EWALLET_KIOSK": "8"
}
```

- **paymentOrigin (str)**:

```
{
 "CASH": "1",
 "CHECK": "2",
 "CREDIT_CARD_ONLINE": "4",
 "CREDIT_CARD": "5",
 "LUNCHEON_VOUCHER": "6",
 "TIME_CHEF": "7",
 "OTHER": "3"
}
```

- **paymentOriginMetaData (str):** (optional) can be used for adding metadata information for the transaction

- **amount (str)**: string of the integer amount of the credit in euro cents (2€ => “200”)

Affluence

Get the number of orders between two dates

GET
https://api.innovorder.fr/orders/count?brandId=<brandId>&startDate=<startDate>&endDate=<endDate>

QueryParams:

- **brandId (mandatory if no customerId or restaurantsId)(number):** filter the count on a brand.
- **customerId (mandatory if no brandId or restaurantsId)(number):** filter the count on a customer.
- **restaurantsId (mandatory if no brandId or customerId)(string):** filter the count on specific restaurants: ids separate by “,”.
- **startDate (optional - must be provided with endDate)(ISO 8601 Date):** date from which orders are counted
- **endDate (optional - must be provided with startDate)(ISO 8601 Date):** date until which orders are counted

Headers:

- Authorization: Bearer <token>

Response:

```
{
    "status": 200,
    "code": "order_succeed",
    "message": "You can access these orders.",
    "data": {
        "count": 25253
    }
}
```

Get the number of orders by interval between two dates

GET
https://api.innovorder.fr/orders/count_by_interval?brandId=<brandId>&startDate=<startDate>&endDate=<endDate>&interval=<interval>

QueryParams:

- **brandId (mandatory if no restaurantId) (number)**: filter the counts on the orders of a brand
- **restaurantId (mandatory if no brandId) (number)**: filter the counts on the orders of a restaurant
- **startDate (mandatory)(ISO 8601 Date)**: date from which orders are counted
- **endDate (mandatory)(ISO 8601 Date)**: date until which orders are counted
- **interval (mandatory)(number)**: size in second of the intervals

Headers:

- Authorization: Bearer <token>

Response:


```
{
    "status": 200,
    "code": "count_order_success",
    "message": "Order counts found",
    "data": [
        {
            "startDate": "2019-11-03T17:29:00.000Z",
            "endDate": "2019-11-05T17:29:00.000Z",
            "count": 3
        },
        {
            "startDate": "2019-11-05T17:29:00.000Z",
            "endDate": "2019-11-07T17:29:00.000Z",
            "count": 10
        },
        {
            "startDate": "2019-11-07T17:29:00.000Z",
            "endDate": "2019-11-09T17:29:00.000Z",
            "count": 9
        },
        {
            "startDate": "2019-11-11T17:29:00.000Z",
            "endDate": "2019-11-13T17:29:00.000Z",
            "count": 4
        },
        {
            "startDate": "2019-11-13T17:29:00.000Z",
            "endDate": "2019-11-15T17:29:00.000Z",
            "count": 18
```

```
        },
        {
            "startDate": "2019-11-17T17:29:00.000Z",
            "endDate": "2019-11-19T17:29:00.000Z",
            "count": 11
        },
        {
            "startDate": "2019-11-19T17:29:00.000Z",
            "endDate": "2019-11-20T17:31:00.000Z",
            "count": 2
        }
    ]
}
```


Note: If there is no order in an interval, it is not in the array.

Order Management System (Web+Kiosk)

Order constants

Consumption modes

```
1. {
2.     "MODE_DELIVERY": 1,
3.     "MODE_TAKE_AWAY": 2,
4.     "MODE_SIT_IN": 3,
5.     "MODE_DRIVE": 4
6. }
```

Order sources

```
1. {
2.     "KIOSK": 1,
3.     "WEB": 2,
4.     "POS": 5
5. }
```

Order main status

```
1. {
2.   "ORDER_PLACED": 0,
3.   "ORDER_VALIDATED": 1,
4.   "ORDER_PREPARING": 2,
5.   "ORDER_PREPARED": 3,
6.   "ORDER_PROCESSED": 4,
7.   "ORDER_CANCELLED": 5,
8.   "ORDER_REFUSED": 6,
9.   "ORDER_COULDNT_BE_PROCEEDED": 7
10. }
```

Order payment status

```
1. {
2.   "PAYMENT_PENDING": 0,
3.   "PAYMENT_OK": 1,
4.   "PAYMENT_REFUSED": 2,
5.   "PAYMENT_REFUNDED": 3
6. }
```

Order delivery status

```
1. {
2.   "DELIVERY_QUOTE_OPEN": 0,
3.   "DELIVERY_QUOTE_EXPIRED": 1,
4.   "DELIVERY_CANCELLED": 2,
5.   "DELIVERY_SEARCH": 3,
6.   "DELIVERY_PICKING": 4,
7.   "DELIVERY_DELIVERING": 5,
```

```
8.   "DELIVERY_DELIVERED": 6
9. }
```

Order model

```
orderId: {
    type: DataTypes.BIGINT,
        allowNull: false,
    primaryKey: true,
    autoIncrement: true,
},
brandId: {
    type: DataTypes.BIGINT,
        allowNull: false,
},
restaurantId: {
    type: DataTypes.BIGINT,
        allowNull: false,
},
channelId: {
    type: DataTypes.BIGINT,
        allowNull: false,
},
menuId: {
    type: DataTypes.BIGINT,
        allowNull: false,
},
promocodeId: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
customerId: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
userId: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
userName: {
    type: DataTypes.STRING,
        allowNull: true,
},
addressRawData: {
    type: DataTypes.TEXT('long'),
        allowNull: true,
```

```
},
addressId: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
cardId: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
paymentMean: {
    type: DataTypes.STRING,
        allowNull: true,
},
ticketNumber: {
    type: DataTypes.STRING(42),
        allowNull: false,
    unique: true,
},
offlineTicketNumber: {
    type: DataTypes.STRING(42),
        allowNull: true,
    unique: false,
},
comment: {
    type: DataTypes.TEXT,
        allowNull: true,
},
currency: {
    type: DataTypes.STRING,
        allowNull: false,
},
locale: {
    type: DataTypes.STRING,
        allowNull: false,
},
expectedDate: {
    type: DataTypes.DATE,
        allowNull: false,
},
expectedDateEnd: {
    type: DataTypes.DATE,
        allowNull: true,
},
mainStatus: {
    type: DataTypes.INTEGER,
        allowNull: false,
},
```

```
driveStatus: {
    type: DataTypes.ENUM,
    values: ORDER_DRIVE_STATUS,
        allowNull: true,
},
deliveryStatus: {
    type: DataTypes.INTEGER,
        allowNull: true,
},
paymentStatus: {
    type: DataTypes.INTEGER,
        allowNull: true,
},
consommationMode: {
    type: DataTypes.INTEGER,
        allowNull: false,
},
forceOnlinePayment: {
    type: DataTypes.BOOLEAN,
        allowNull: false,
    defaultValue: false,
},
deliveryFee: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_delivery_fee',
        },
    },
},
minimumAmountDelivery: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_minid',
        },
    },
},
minimumAmountForFreeDelivery: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
```

```
            args: [0],
            msg: 'invalid_parameters_miniFree',
        },
    },
},
ttcTotalPrice: {
    type: DataTypes.BIGINT,
        allowNull: false,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_ttc',
        },
    },
},
vatTotalPrice: {
    type: DataTypes.BIGINT,
        allowNull: false,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameter_vat',
        },
    },
},
ttcUpselling: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_upselling',
        },
    },
},
ttcEntranceFee: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_ttcEntranceFee',
        },
    },
},
ttcServiceFee: {
    type: DataTypes.BIGINT,
```

```
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_serviceFee',
        },
    },
},
ttcDeliveryFee: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_ttcDeliveryFee',
        },
    },
},
ttcPromocodeReduction: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_promocode',
        },
    },
},
ttcGrantReduction: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_grant_reduction',
        },
    },
},
takeAwayReduction: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_take_away_reduc',
        },
    },
```

```
},
ttcTakeAwayReduction: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameters_ttc_take_away_reduc',
        },
    },
},
ttcRoyaltyReduction: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
emittedRoyalties: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
ewalletBalanceBeforeOrder: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
ewalletBalanceAfterOrder: {
    type: DataTypes.BIGINT,
        allowNull: true,
},
entranceFeeRawData: {
    type: DataTypes.TEXT('long'),
        allowNull: true,
},
grantRawData: {
    type: DataTypes.TEXT('long'),
        allowNull: true,
},
pricingRuleRawData: {
    type: DataTypes.TEXT('long'),
        allowNull: true,
},
cartRawData: {
    type: DataTypes.TEXT('long'),
        allowNull: false,
},
orderRecapRawData: {
    type: DataTypes.TEXT('long'),
        allowNull: false,
},
```

```
deliveryType: {
    type: DataTypes.ENUM,
    values: ORDER_DELIVERY_TYPE,
        allowNull: true,
},
servingCount: {
        type: DataTypes.BIGINT,
    defaultValue: 1,
},
cancellationReason: {
        type: DataTypes.ENUM,
    allowNull: true,
    values: Object.keys(ORDER_CANCELLATION_REASON),
},
cancellationReasonDescription: {
        type: DataTypes.STRING,
    allowNull: true,
    defaultValue: null,
},
externalPosOrderId: {
        type: DataTypes.STRING,
    allowNull: true,
    defaultValue: null,
},
ttcOnlinePaymentDiscount: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameter_ttc_online_payment_reduction',
        },
    },
},
onlinePaymentDiscount: {
    type: DataTypes.BIGINT,
        allowNull: true,
    validate: {
        min: {
            args: [0],
            msg: 'invalid_parameter_online_payment_reduction',
        },
    },
},
```

Corresponding enums:

```
mainStatus: {
```

```
        0: "ORDER_PLACED",
        1: "ORDER_VALIDATED",
        2: "ORDER_PREPARING",
        3: "ORDER_PREPARED",
        4: "ORDER_PROCESSED",
        5: "ORDER_CANCELLED",
        6: "ORDER_REFUSED",
        7: "ORDER_COULDNT_BE_PROCEEDED",
        8: "ORDER_AWAITING_ACTION",
},
driveStatus: [
        "DRIVE_INIT",
        "CUSTOMER_ARRIVED",
        "RECEPTION_HAS_SEEN"
],
deliveryStatus: {
        0: "DELIVERY_QUOTE_OPEN",
        1: "DELIVERY_QUOTE_EXPIRED",
        2: "DELIVERY_CANCELLED",
        3: "DELIVERY_SEARCH",
        4: "DELIVERY_PICKING",
        5: "DELIVERY_DELIVERING",
        6: "DELIVERY_DELIVERED",
},
deliveryType: [
        "OWN_DELIVERY",
        "STUART_V1",
        "STUART_V2"
],
paymentStatus: {
        0: "PAYMENT_PENDING",
        1: "PAYMENT_OK",
        2: "PAYMENT_REFUSED",
        3: "PAYMENT_REFUNDED",
},
consommationMode: (int) {
        1: "MODE_DELIVERY",
        2: "MODE_TAKE_AWAY",
        3: "MODE_SIT_IN",
        4: "MODE_DRIVE",
},
cancellationReason: [
        "CANNOT_FULFILL_ORDER",
        "ORDERING_MISTAKE",
        "UNSATISFIED_CUSTOMER",
        "CUSTOMER_NO_SHOW",
        "CUSTOMER_LEAVES",
```

```
        "LEARNING_MODE",
        "TECHNICAL",
        "OTHER"
]
```

Preview an order

POST https://api.innovorder.fr/orders/preview

Headers:

- Authorization: Bearer <token>

Body:

```
{
        restaurantId: number,
        channelId: number, // only 2 (for webordering) is currently supported
        consumptionMode: ConsumptionModeType("MODE_DELIVERY" |
"MODE_TAKE_AWAY" | "MODE_SIT_IN" | "MODE_DRIVE")
        menuId: number,
        cart: ProductDTO[],
        customerId?: number,
        expectedAt?:Date, // modulo 15 min
        expectedAtEnd?: Date,
        payments?: PaymentDTO[],
        royaltyDiscountValue?: number,
        promocode?: string,
        comment?: string,
        addressId?: number,
        tableName?: string,
}
```

with

```
ProductDTO {
        productId: number,
        quantity: number,
        steps: StepDTO[]
}

StepDTO {
        stepId: number,
        products: ProductDTO[]
}
```

```
PaymentDTO {
      paymentMethodId: number,
      amount: number,
      quantity: number,
      metadata?: string
}
```

Response:

```
{
   status:200,
   code:"order_parameters_validated",
   message:"Vos paramètres de commande sont valides.",
   data: ValidatedOrder
}
```

with

```
ValidatedOrder = {
      ...bodyOfTheRequest,
      brandId: number;
      currency: string;
      cart: EnrichedProduct[];
      recap: OrderRecap;
}

EnrichedProduct = {
    productId: number;
    quantity: number;
    steps: EnrichedStep[];

    name: string;
    price: number; // quantity * (unitPrice + sum of the childProducts
prices)
    originalUnitPrice: number; // price of the product alone in the menu
    unitPrice: number; // price of the product in the menu if product not in
a step
    // price defined in the step if the product is in a step and
totalQuantity < maxChoice
    // exceedPrice defined in the step if the product is in a step and
totalQuantity > maxChoice
    vat: number; // 10000 => 100%
    externalReferences: { [provider: string]: string };
    tags: {
       [tagId: number]: {
          tagId: number;
          name: string;
       }
    },
```

```
   servingCount: number; // The number of customers attached to the product
};

EnrichedStep = {
    stepId: number;
    name: string;
    ticketLabel: string;
    products: EnrichedProduct[];
};


OrderRecap = {
    currency: string;
    totalPrice: PriceBreakdown;
    taxRatesBreakdown: TaxRateBreakdown[];
    discounts: DiscountRecap[];
    fees: FeeRecap[];
    products: ProductRecap[];
    globalDiscount: GlobalDiscount;
};

 ProductRecap = {
    label: string;
    id: number;
    taxRate: number;
    totalPrice: PriceBreakdown;
    unitPrice: PriceBreakdown;
    depth: number;
    quantity: number;
    additionalData?: any;
};

FeeRecap = {
    label: string;
    id: string | number;
    value: number;
    taxRate: number;
    type: string;
    totalPrice: PriceBreakdown;
    additionalData?: any;
};

GlobalDiscount = {
    discountRate: number;
    promocode?: {
        code: string;
        value: number;
```

```
    };
};

DiscountRecap = {
    label: string;
    id: string;
    amount: number;
    taxRatesBreakdown?: TaxRateBreakdown[];
    strategy: {
        type: DISCOUNT_TYPE;
        applyMethod: DiscountApplyMethod;
        value: number;
        scope: {
            type: DISCOUNTABLE_TYPE[];
            filter: { type: DiscountFilterType; value: (string | number)[]
};
        };
    };
    additionalData?: any;
};

TaxRateBreakdown = {
    taxRate: number;
    totalPrice: PriceBreakdown;
};

PriceBreakdown = {
    totalInclTax: number;
    totalExclTax: number;
    totalTax: number;
    totalInclTaxWithDiscount: number;
    totalExclTaxWithDiscount: number;
    totalTaxWithDiscount: number;
};

enum DISCOUNT_TYPE {
    PROMOCODE = 'PROMOCODE',
    LOYALTY = 'LOYALTY',
    GRANT = 'GRANT',
    TAKE_AWAY_REDUCTION = 'TAKE_AWAY_REDUCTION',
    COMPENSATION = 'COMPENSATION',
    POS = 'POS',
    FORCE_TOTAL_AMOUNT = 'FORCE_TOTAL_AMOUNT',
    ONLINE_PAYMENT = 'ONLINE_PAYMENT',
}

enum DiscountApplyMethod {
```

```
    RATE = 'RATE',
    GRANT_ABSOLUTE_GLOBAL = 'GRANT_ABSOLUTE_GLOBAL',
    ABSOLUTE_CART = 'ABSOLUTE_CART',
    ABSOLUTE_GLOBAL = 'ABSOLUTE_GLOBAL',
    FORCE_TOTAL_AMOUNT = 'FORCE_TOTAL_AMOUNT',
}

enum DiscountFilterType {
    MANUAL = 'MANUAL',
    TAGS = 'TAGS',
}
```

⇒ Gestion du paiement dans la preview

le field *payments* dans la preview n'est pas obligatoire (auquel cas un simple tableau vide suffit), ne pas renseigner de moyen de paiement empêche cependant le backend de prendre en compte des potentielles réductions liés à un moyen de paiement spécifique.

Pour prendre en compte toutes les réductions liés à un moyen de paiement, il convient de renseigner le *paymentMethodId* sans le field *amount*, de la manière suivante :

```
"payments": [

      {

          "paymentMethodId": "3737"

      }

  ],
```

⇒ Reconstituer un panier

1. Utiliser le cart pour afficher les produits et leur prix

   ⚠️ *Ne pas utiliser les ProductRecap du recap, ils correspondent à la vue administrative du panier avec les prix des formules partagées sur les produits qui les composent*

2. Utiliser recap.fees pour afficher les admissions

3. Utiliser recap.discounts pour afficher les subventions et les autres réductions

4. Utiliser recap.totalPrice.totalInclTaxWithDiscount pour afficher le total du panier

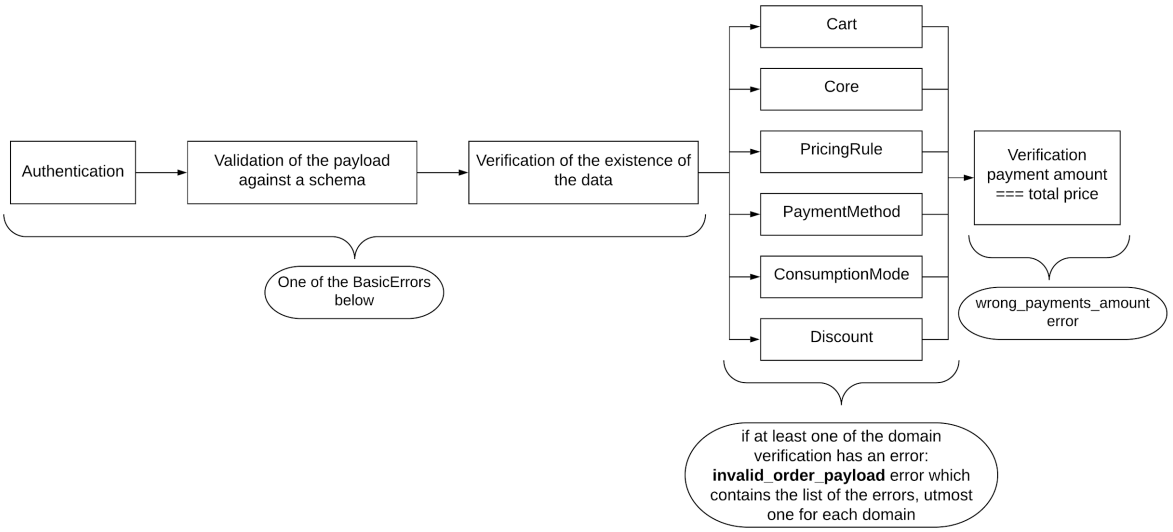
⇒ **Prévisualisation des prix et régimes tarifaires**

Lors d'une requête de prévisualisation (`preview`) sans `customerId`, l'application d'un régime tarifaire reste possible dans certains cas :

- Si le canal de vente est configuré en mode hybride dans le backoffice, un régime tarifaire par défaut peut être appliqué même sans identification client.
- Si un compte client externe de type "comptant/cash" est spécifié dans la requête, le régime tarifaire associé à ce compte sera appliqué. Dans ce cas :
  - Les admissions liées à ce compte seront reflétées dans `recap.fees`
  - Les subventions liées à ce compte seront reflétées dans `recap.discounts`

Order preview errors

```
type InnovorderErroredResponse {
    status: number; // ex: 400
    code: string; // ex: "missing_parameters",
    message: string; // ex: "Paramètres manquants."
    extraData?: object | array;
}
```



Basic Errors

- **permission_denied 401**
  status: 401
  code: permission_denied
  extraData: none

  Signification:

  - Pas de header Authorization: Bearer <token>
  - Token invalide, impossible d'identifier l'utilisateur (user ou customer)
  - L'utilisateur authentifié n'est pas un customer ou une brand (ou un compte de service quand cela sera implémenté)

- **permission_denied 403**
  status: 403
  code: permission_denied
  extraData: non

  Signification:
  - L'utilisateur authentifié (user ou customer) essaie de créer/preview une commande pour un customer auquel il n'a pas accès (un customer d'une autre brand si user, un customer qui n'est pas lui même si customer)

- **invalid_parameters**
  status: 400
  code: invalid_parameters
  extraData: string[]: description des erreurs

  Signification:
  - la requête ne respecte pas le schéma

- **restaurant_not_found**
  status: 400
  code: restaurant_not_found
  extraData: { restaurantId }

  Signification:
  - le restaurantId du payload ne correspond à aucun restaurant existant

- **customer_not_found**
  status: 400
  code: customer_not_found
  extraData: { customerId }

  Signification:
  - le customerId du payload ne correspond à aucun customer existant

- **menu_not_found**
  status: 400
  code: menu_not_found
  extraData: { menuId }

  Signification:
  - le menuId du payload ne correspond à aucun menu existant

- **payment_method_not_found**
  status: 400
  code: payment_method_not_found
  extraData: { paymentMethodId }

  Signification:
  - un paymentMethodId d'un payment du payload ne correspond à aucune paymentMethod existante

- **address_not_found**
  status: 400
  code: address_not_found
  extraData: { addressId }

  Signification:
  - le addressId du payload ne correspond à aucune address existante

- **consumption_mode_not_activated**
  status: 403
  code: consumption_mode_not_activated
  extraData: { restaurantId, consumptionModeType, channelId }

Signification:

- Le mode de consommation n’est pas activé pour ce restaurant et ce channel. C’est probablement un problème de configuration

- **invalid_order_payload**
  status: 400
  code: invalid_order_payload
  extraData: { errors: InnovorderErroredResponse[] }

  Signification:
  - la validation du panier a échoué. Vous trouverez plus de détails dans la liste d’erreur d’extraData, Il peut y avoir une erreur par domain: Cart, Core (Authorization d’accès), PaymentMethod, Schedule, ConsumptionMode, Discount.

Core Errors

- **permission_denied 403**
  status: 403
  code: permission_denied
  extraData: non

  Signification:
  - L'utilisateur authentifié (user ou customer) essaie de créer/preview une commande pour un customer auquel il n'a pas accès (un customer d'une autre brand si user, un customer qui n'est pas lui même si customer)

- **incompatible_customer_and_restaurant**
  status: 400
  code: incompatible_customer_and_restaurant
  extraData: { customerId, restaurantId }

  Signification:
  - le customer et le restaurant ne sont pas dans la même brand

- **incompatible_menu_and_restaurant**
  status: 400
  code: incompatible_menu_and_restaurant
  extraData: { menuId, restaurantId }

Signification:
  - le menu n'appartient pas au restaurant

Cart Errors

⚠️ If the cart is invalid, you will receive this error:

- **invalid_cart**
  status: 400
  code: invalid_cart
  extraData: { status: “VALID” | “INVALID”, products: ValidationProduct[] }

  Signification:
  - Something is wrong with this cart. Consult the extraData to know what. The ExtraData is a ValidationCart:

```
export enum ValidationStatus {
   Valid = 'VALID',
   Invalid = 'INVALID',
}
export type ValidationCart = {
   status: ValidationStatus;
   products: ValidationProduct[];
};

export interface ValidationStep {
   stepId: number;
   status: ValidationStatus;
   reasons: InnovorderErroredResponse[];
   products: ValidationProduct[];
}

export interface ValidationProduct {
   productId: number;
   quantity: number;
   status: ValidationStatus;
   reasons: InnovorderErroredResponse[];
   steps: ValidationStep[];
}
```

The reasons are a list of errors among the following:

- **product_not_in_menu**
  status: 400
  code: product_not_in_menu

extraData: { productId, menuId }

Signification:
  - le product n'appartient pas au menu

- **step_not_in_menu**
  status: 400
  code: step_not_in_menu
  extraData: { stepId, menuId }

  Signification:
  - la step n'appartient pas au menu

- **product_not_available_in_consumption_mode**
  status: 400
  code: product_not_available_in_consumption_mode
  extraData: { productId, consumptionMode }

  Signification:
  - le product n'est pas disponible dans le mode de consommation consumptionMode

- **product_not_available_on_web**
  status: 400
  code: product_not_available_on_web
  extraData: { productId }

  Signification:
  - le product n'est pas disponible pour la commande en ligne

- **wrong_number_of_steps**
  status: 400
  code: wrong_number_of_steps
  extraData: { productId, stepsLength, requiredStepsLength }

  Signification:
  - le product a été envoyé avec des steps manquantes (si un produit a des steps elles doivent toutes être renseignées)

- **step_not_in_product**
  status: 400
  code: step_not_in_product
  extraData: { productId, stepId }

  Signification:
  - la step ne fait pas partie des steps du product (si un produit a des steps elles doivent toutes être renseignées)

- **product_not_in_step**
  status: 400
  code: product_not_in_step
  extraData: { productId, stepId }

  Signification:
  - le product ne fait pas parti des products possible dans la step

- **not_enough_products_in_step**
  status: 400
  code: not_enough_products_in_step
  extraData: { stepId, minChoice, productsQuantity }

  Signification:
  - La somme des quantités des products de la step est plus petit que son minChoice

- **too_many_products_in_step**
  status: 400
  code: too_many_products_in_step
  extraData: { stepId, maxChoice, productsQuantity }

  Signification:
  - La somme des quantités des products de la step est plus grande que son maxChoice ET les excès ne sont pas autorisés sur la step (isExcessAllowed)

- **too_many_products**
  status: 400
  code: too_many_products
  extraData: { stepId, productId, maxQuantity, quantity }

Signification:
  - La verification des quantités maximales est activée sur la step (isQuantitySelectionEnabled) ET la somme des quantités du product sur la step est plus grande que son maxProductQuantity

- **weight_not_supported**
  status: 400
  code: weight_not_supported
  extraData: { stepId }

  Signification:
  - Le calcul par poids est activée sur la step (isWeightStrategyEnabled) (et ce n'est pas supportée par cette route)

- **insufficient_quantity_product**
  status: 400
  code: insufficient_quantity_product
  extraData: { productId, currentQuantity }

  Signification:
  - La somme des quantités du product sur l'ensemble du panier est plus grand que sa quantité disponible en restaurant (currentQuantity)

PaymentMethod Errors

- **payment_method_not_activated**
  status: 403
  code: payment_method_not_activated
  extraData:{ paymentMethodId, consumptionModeId }

  Signification:

  - Le moyen de paiement n’est pas activé pour ce mode de consommation. Le moyen de paiement existe mais n’est pas relié au bon mode de consommation. Cela provient surement d’une erreur lors de la récupération du paymentMethodId

- **payment_method_not_supported**
  status: 403

code: payment_method_not_supported
extraData: { paymentMethodId, paymentMethodCode }

Signification:

- Le moyen de paiement n’est pas supporté par la route. Actuellement, seul les paiements externes sont autorisés

- **wrong_payments_amount**
  status: 403
  code: wrong_payments_amount
  extraData: { amountToPay, amountPaid }

  Signification:

  - La somme des montants des moyens de paiements (amountPaid) n’est pas égale au prix de la commande (amountToPay). Les paiements partiels ne sont pas autorisés.

    Utilisez la route /orders/preview sans le champs payments pour valider la commande sans vérification des paiements et obtenir son prix

Schedule Errors

- **invalid_schedule**
  status: 400
  code: invalid_schedule
  extraData: { message, ... }

  Signification:

  - Cette erreur legacy est utilisée pour plusieurs raisons, cf le message dans extraData pour la comprendre.

- **module_not_setup**
  status: 403
  code: module_not_setup
  extraData: { module, restaurantId, channelId }

  Signification:

    - C’est un problème de configuration du restaurant. Adressez vous à Innovorder.

- **can_not_order_now**
  status: 403
  code: can_not_order_now
  extraData: { date, schedules, timeZoneId }

  Signification:

    - La commande ne pas être passée à ce moment. Cela arrive si la précommande est désactivée et que now() n’est pas dans un créneau d’horaire d’ouverture de la commande en ligne

- **can_not_order_now**
  status: 403
  code: can_not_order_now
  extraData: { date, schedules, timeZoneId }

  Signification:

    - La commande ne pas être passée à ce moment. Cela arrive si la précommande est désactivée et que now() n’est pas dans un créneau d’horaire d’ouverture de la commande en ligne

- **forbidden_expected_at**
  status: 403
  code: forbidden_expected_at
  extraData: { expectedAt, message }

  Signification:

    - La commande ne pas être passée pour un autre jour qu’aujourd’hui. Cela arrive si la précommande est désactivée et que expectedAt n’est pas pour aujourd’hui
    - La commande ne peux pas être passée en dehors d’un créneau de collecte (ou de livraison dans le cas de la livraison avec des créneaux spéciaux)

- **time_slot_too_early**
  status: 403
  code: time_slot_too_early
  extraData: { expectedAtEnd, now, delay }

  Signification:

  - now + delay <= expectedAtEnd. Delay = rushDelay + consumptionModeDelay
- **expected_at_too_early**
  status: 403
  code: expected_at_too_early
  extraData: { expectedAt, now, delay }

  Signification:

  - now + delay <= expectedAt. Delay = rushDelay + consumptionModeDelay

Discount Errors

- **promocode_not_found**
  status: 400
  code: promocode_not_found
  extraData: { promocode }

  Signification:

  - Le promocode fourni n’existe pas

- **promocode_start_date_is_in_the_future**
  status: 400
  code: promocode_start_date_is_in_the_future
  extraData: { promocode, startDate }

  Signification:

  - Le promocode n’est pas encore valide

- **royalty_not_active_in_restaurant**
  status: 400

code: royalty_not_active_in_restaurant
extraData: { restaurantId }

Signification:

- Ce restaurant n'a pas activé la fidélité

- **not_enough_royalty_balance**
  status: 400
  code: not_enough_royalty_balance
  extraData: { restaurantId }

  Signification:

  - Vous n'avez pas assez sur votre programme fidélité
  -

- **are_all_promocodes_already_used**
  status: 400
  code: are_all_promocodes_already_used
  extraData: { promocodeId, quantity }

  Signification:

  - La quantité maximum d'utilisations de ce code promo est atteinte

- **has_customer_already_used_promocode**
  status: 400
  code: has_customer_already_used_promocode
  extraData: { promocodeId, customerId }

  Signification:

  - Le consommateur a déjà utilisé ce promocode

Create an order

POST https://api.innovorder.fr/orders

Headers:

- Authorization: Bearer <token>
- idempotency-key: uuid (to prevent the double creation of an order)

Body:

```
// Same as preview but with customerId, payments and expectedAt required
{
      restaurantId: number,
      channelId: number, // only 2 (for webordering) is currently supported
      consumptionMode: ConsumptionModeType("MODE_DELIVERY" |
"MODE_TAKE_AWAY" | "MODE_SIT_IN" | "MODE_DRIVE")
      menuId: number,
      cart: ProductDTO[],
      customerId: number,
      expectedAt:Date, // modulo 15 min
      expectedAtEnd?: Date,
      payments: PaymentDTO[],
      royaltyDiscountValue?: number,
      promocode?: string,
      comment?: string,
      addressId?: number,
      tableName?: string,
}
```

Response:

```
{
   status:200,
   code:"order_created",
   message:"Vos paramètres de commande sont valides.",
   data: Order // the same as GET /orders/<orderId> + some other fields
}
```

This route create the order, an associated transaction and the associated payment transactions (one for each payment method).

The payments are registered in the currency of the restaurant

Couponing

Create a coupon

Backoffice users can create coupons, we offer various kind of coupons:

- Discount in value
- Discount in percentage

The coupons can be applied using the following strategies:

- Enable on all modes or specific modes (drive, take away, delivery, sit in)
- Enable on all restaurants or specific one
- A discount can be enabled at a starting date (with or without ending date)
- You can limit the number of global use
- You can specify if a discount is limited to one usage per account
- You can specify a minimum amount

POST https://api.innovorder.fr/promocodes

Params :

- brandId – Id of the brand
- code – String: the code for the end-users
- consumptionFlags:
  - Delivery only, use 1
  - Take away only, use 2
  - SitIn only, use 4
  - Drive only, use 8
  - If you want multiple consumption mode, we use binary flag, so for enabling all modes do: (((1 | 2) | 4) | 8) = 15 ; value to send should be **15**
- endDate: Datetime, e.g: "2019-11-30T23:00:00.000Z"
- isUnique: true
- minimumAmount: value in cents
- noLimit: true or false
- quantity: an integer
- reductionType: “absolute” or “percentage”
- reductionValue: If absolute: value in cents. If percentage, value should be % x 100 (E.g: 5% = 500 in value)
- restaurantIds: Array of restaurantId, e.g: ["2"]
- startDate: Datetime, e.g: "2019-07-29T14:47:36.902Z"

Use a coupon on webordering

GET https://api.innovorder.fr/promocodes/validate

Description

Check if a coupon is valid in a given context. You need to be authenticated as a customer.

Query String

- amount – Integer, amount in cents
- code – String, the code to validate
- consumption_mode – Integer, the int value of a consumption_mode (e.g: 2 for take away)
- restaurant_id – Integer, the restaurantId

For using the coupon on webordering, add a promocodeId and promocode object to POST /orders.

Overview Analytics

GET https://api.innovorder.fr/analytics/overview

Description
Get the analytics overview (JSON) from the home page of the back-office. The API provides the following data:

- **userCount –** Integer: *new users for params*
- **orderCount –** Integer: *count of the orders for the params*
- **orderSum –** Integer: *sum of the orders (tax included) for the params*
- **orderAvg –** Integer: *average basket on the orders (tax included)*
- **orderCountLine –** Array of {x: int, y:int} object: *used for drawing the charts, x is the value and y the timestamp of the corresponding aggregation*
- **orderSumLine –** Array of {x: int, y:int} object: *used for drawing the charts, x is the value and y the timestamp of the corresponding aggregation*

- **orderAvgLine –** Array of {x: int, y:int} object: *used for drawing the charts, x is the value and y the timestamp of the corresponding aggregation*

Params – Query String

- **restaurantIds -** String: restaurantId join by a comma (e.g: “192,122”)
- **channelIds -** String: channelId join by a comma (e.g: “1,2”)
- **modes -** String: consumptionMode join by a comma (e.g: “1,2”)
- **start -** Date ISO format: start date of the interval
- **end -** Date ISO format: end date of the interval

Example output


```
{
   "status":200,
   "code":"analytic_succeed",
   "message":"Les analytiques ont été générés avec succès.",
   "data":{
      "userCount":8,
      "orderCount":3,
      "orderSum":7200,
      "orderAvg":2400,
       "orderCountLine":[
         {
            "y":0,
            "x":1512345600
         },
         {
            "y":1,
            "x":1512432000
         },
         ...
       ],
       "orderSumLine":[
         {
            "y":0,
            "x":1512345600
         },
         {
            "y":3100,
            "x":1512432000
         },
         ...
       ],
       "orderAvgLine":[
         {
            "y":0,
            "x":1512345600
         },
         {
            "y":3100,
            "x":1512432000
         },
         ...
       ],
       "userCountLine":[
         {
            "y":0,
            "x":1512345600
         },
```

```
        ...
      ]
    }
}
```

Note: Prices are defined in cents, you should divide by 100 for EUR currency.

Online Ordering API

Overview

This API provides a workflow for handling online orders, allowing applications to retrieve configuration, menu details, consumption modes, order previews, and track orders in real time.

Base URL

https://api.innovorder.fr

Authentication

To access the API, include an Authorization header with a Bearer token. Use /oauth/login to retrieve the token if not already available.

Step 1: Retrieve Web Ordering Configuration

Endpoint

GET /brands/webordering-configuration/<brandHash>

Description

Fetches the configuration for a specific brand's web ordering setup.

Parameters

- brandHash *(string)*: Unique identifier for the brand.

Response

- JSON object with configuration details, such as available ordering features, payment options, and additional settings.

Example Request

GET
https://api.innovorder.fr/brands/webordering-configuration/{brandHash}

Step 2: Retrieve Menu Hash Map

Endpoint

GET /menus/<menuid>/hash_map

Description
Fetches a compressed version of the menu, including products, categories, and steps.

Parameters

- menuid *(string)*: Menu identifier.

Response

- JSON object with the menu structure (e.g., products, categories, and steps).

Example Request

GET https://api.innovorder.fr/menus/{menuid}/hash_map

Step 3: Retrieve Consumption Modes

Endpoint

GET /consumption_modes?channel_id=2&restaurant_id=<restaurantId>

Description
Fetches the available consumption modes for a restaurant on a specified channel.

Parameters

- channel_id *(integer)*: For web ordering, use 2.
- restaurant_id *(string)*: Identifier for the restaurant.

Response

- Array of available consumption modes.

Example Request

GET https://api.innovorder.fr/consumption_modes?channel_id=2&restaurant_id={restaurantId}

Step 4: Preview Order

Endpoint

POST /orders/preview

Description
Provides an order preview, allowing verification of items and pricing before finalizing the order.

Headers

- Authorization: Bearer <token>
- Content-Type: application/json

Body Parameters *(JSON)*

- restaurant_id *(string)*: Identifier for the restaurant.
- products *(array)*: List of items with product_id, quantity, and optional steps

Response

- JSON object with a validated order, showing detailed breakdowns (e.g., prices, taxes).

Example Request

See previous documentation.

Step 5: Create Order

Endpoint

POST /orders

Description
Creates the order after user confirmation.

Headers

- Authorization: Bearer <token>
- Content-Type: application/json

Body Parameters *(JSON)*

- restaurant_id *(string)*: Unique identifier for the restaurant.

- products *(array)*: List of products.
- consumption_mode_id *(integer)*: Selected consumption mode.

Response

- Order confirmation and estimated preparation time.

Example Request

See previous documentation.

Step 6: Track Order Status Using Firestore

Description
After order creation, track KDS tickets via Firestore for real-time updates.

Firestore Path

restaurants/<restaurantId>/kds-tickets

Firestore Usage
The kds-tickets collection in Firestore provides live updates on order preparation status.

Example Path

restaurants/12345/kds-tickets

Note
Ensure Firestore has the correct read access and security rules to enable authorized KDS tracking

Notice: The information in this publication is subject to change without notice.

THIS PUBLICATION IS PROVIDED “AS IS” WITHOUT WARRANTIES OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NONINFRINGEMENT. INNOVORDER, SAS (“INNOVORDER”), SHALL NOT BE LIABLE FOR TECHNICAL OR EDITORIAL ERRORS OR OMISSIONS CONTAINED HEREIN, NOR FOR DIRECT, INCIDENTAL, CONSEQUENTIAL OR ANY OTHER DAMAGES RESULTING FROM THE FURNISHING, PERFORMANCE, OR USE OF THIS PUBLICATION, EVEN IF INNOVORDER HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES IN ADVANCE.

This publication contains information protected by copyright. Except for internal distribution, no part of this publication may be photocopied or reproduced in any form without prior written consent from Innovorder.

Product names mentioned herein may be trademarks and/or registered trademarks of their respective companies.

Copyright © 2025 Innovorder SAS